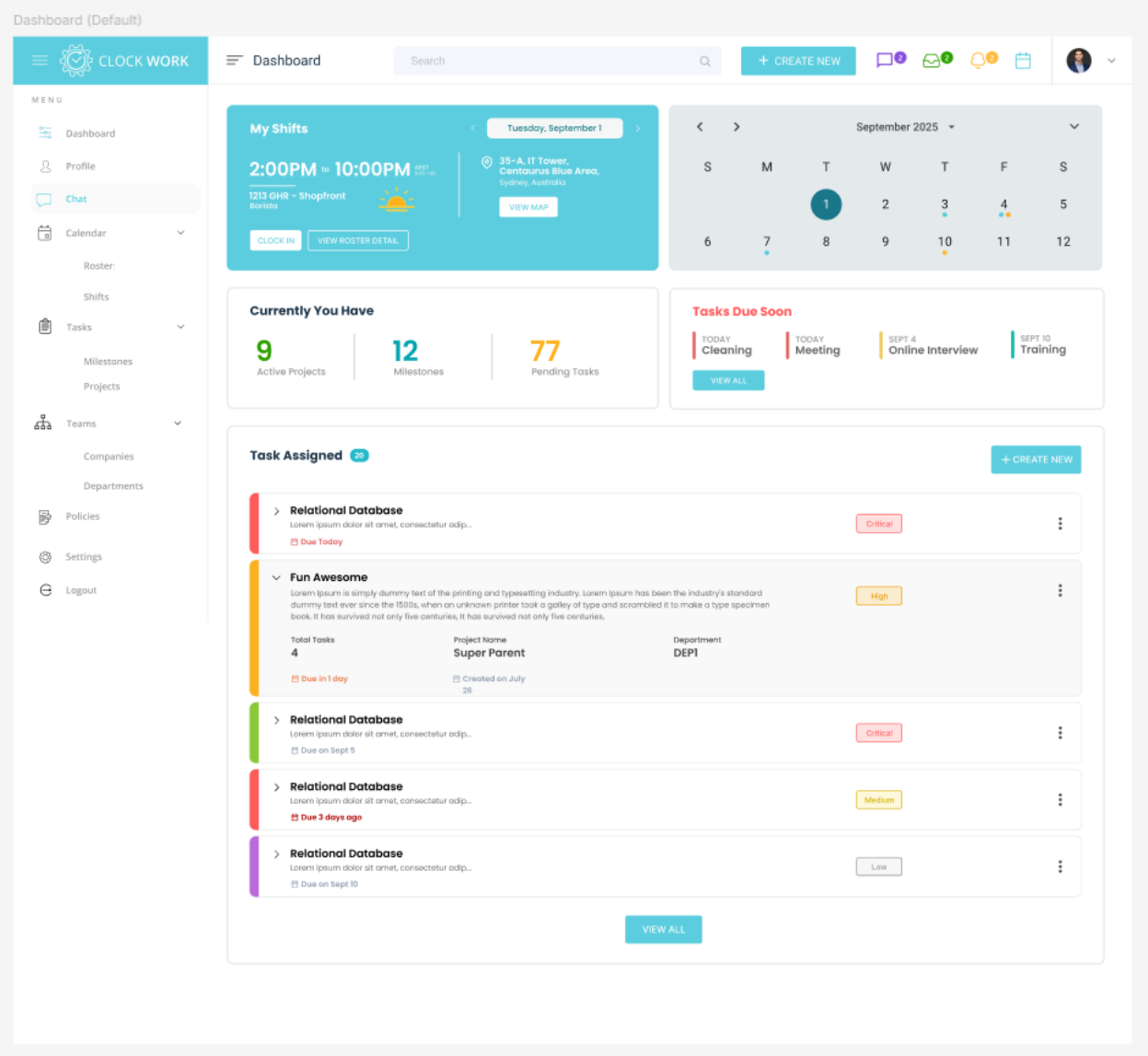
Clockwork Commons Induction

Overview:

Clockwork Commons is a concept designed to aid in the handover of all the Clockwork MVP work.

Figma Designs:



Current Teams & Their Merged Work:

Code Crusaders:

Clockwork Commons is a concept designed to aid in the handover of all the Clockwork MVP work.

teamawesome.fun:

teamawesome.fun's work significantly restructures and expands the repository by introducing a full backend environment and improving documentation.

The README.md file has been rewritten into a detailed setup guide covering both frontend and backend components, including installation and run instructions. The .gitignore file has been updated to exclude build artifacts, environment files, and cache directories for a cleaner project structure.

New configuration files include .nvmrc (specifying Node.js 20.18.2), VS Code workspace settings enforcing Prettier formatting, and a minimal backend README.md. A new docker-compose.yml defines containers for the Node.js app and two PostgreSQL databases, each with its own configuration and persistent storage.

The backend's package.json has been modernized with TypeScript, Jest, Kysely ORM, and other dependencies. Scripts have been added for building, testing, and generating database types. The new permissions-categories.json defines structured role-based permissions for organisation, project, and member management.

The backend source code introduces app.ts, which sets up an Express server with authentication (BetterAuth), CORS, JSON parsing, and route handling. New controllers handle authentication (auth.controller.ts) and audit logs (audit.controller.ts), implementing secure user registration, validation, and structured API responses.

Overall, this commit converts the repository from a frontend-only project into a full-stack application with a TypeScript-based backend, Dockerized environment, clear configuration, and foundational authentication and permissions systems.

All PaperWork Merges

This work introduces a comprehensive restructuring and modernization of the ClockWork PaperWork repository, particularly focusing on backend improvements, database integration, and cleanup of legacy materials. It consolidates several branches, including Paperworks-Merger-all, PaperWork-Module-postgres-cascade, and PaperWork-Module-origin-feature, into a unified and consistent backend framework. The update also removes outdated files, such as archived

documents and earlier version directories, and adds a new pnpm-lock.yaml file to standardize dependency management.

The backend has been significantly enhanced with full PostgreSQL integration and expanded Express API functionality. The cascade_tasks_eps.js file now provides complete CRUD operations for cascade tasks, along with new filtering capabilities and an endpoint to display pending task statistics. The common_blocks_eps.js file has been extended to manage version control for common blocks and automatically generate cascade and external update tasks, ensuring consistency across related data structures.

The create_document_eps.js file introduces a /default-templates endpoint to provide standardized document templates while retaining legacy variable endpoints for backward compatibility. The customise_document_eps.js file has been refactored to align with MySQL syntax conventions, simplify redundant endpoints, and improve request validation and version management during updates. These enhancements improve code clarity, maintainability, and database reliability.

Additionally, the commit removes numerous outdated documentation files and archived materials from the /README/ and /Archives/ directories to streamline the repository. A new backend environment configuration file (.env) has been added, specifying PostgreSQL connection details, including host, port, database name, and credentials, alongside environment settings for development.

In summary, this commit transforms the project into a more maintainable and production-ready system by merging fragmented branches, introducing structured PostgreSQL support, enhancing backend APIs, and removing obsolete assets. The changes establish a cleaner, more consistent foundation for future backend development and integration.